Data Science Masters

Time Series-1 — Assignment Answers

Q1. What is a time series, and what are some common applications of time series analysis?

A time series is a sequence of observations collected or recorded in chronological order, usually at regular time intervals such as hourly, daily, monthly, or yearly. Time series analysis studies the patterns, dependencies, and changes in these observations over time.

Common applications include:

• Sales and demand forecasting in business.

• Stock-market and financial forecasting.

• Weather and climate analysis.

• Electricity-load and energy-demand forecasting.

• Economic indicators such as inflation and GDP.

• Website traffic and customer-activity forecasting.

Q2. What are some common time series patterns, and how can they be identified and interpreted?

The common patterns are:

1. Trend — a long-term upward or downward movement.

2. Seasonality — a pattern that repeats at a fixed and known period, such as higher retail sales every December.

3. Cyclic variation — longer-term rises and falls that do not necessarily have a fixed period.

4. Irregular or random variation — unpredictable fluctuations not explained by the systematic patterns.

These patterns can be identified using line plots, seasonal plots, moving averages, decomposition, and autocorrelation plots. Understanding the pattern helps select an appropriate forecasting model.

Q3. How can time series data be preprocessed before applying analysis techniques?

Typical preprocessing steps include:

• Convert the time column to a proper datetime format and sort observations chronologically.

• Check for missing values and handle them appropriately.

• Detect unusual observations or outliers.

• Aggregate or resample data when necessary.

• Transform the data, for example using a logarithm, when variance increases with the level.

• Check stationarity and use differencing if required.

• Split the data chronologically into training and testing sets; random shuffling should generally be avoided for forecasting.

Q4. How can time series forecasting be used in business decision-making, and what are some common challenges and limitations?

Forecasting can support inventory planning, production scheduling, staffing, budgeting, cash-flow planning, pricing, and demand management. For example, a retailer can forecast future demand to decide how much stock to order.

Challenges include noisy data, missing observations, structural changes, unexpected events, seasonality, model-selection errors, and changing customer behavior. Forecasts are also uncertain, so predictions should be considered estimates rather than guaranteed outcomes.

Q5. What is ARIMA modelling, and how can it be used to forecast time series data?

ARIMA stands for AutoRegressive Integrated Moving Average. It is commonly represented as ARIMA(p,d,q).

• p = order of the autoregressive (AR) component.

• d = number of differences used to make the series stationary.

• q = order of the moving-average (MA) component.

ARIMA models the relationship between an observation and its previous observations and past forecast errors. After selecting suitable p, d, and q values, the model is fitted on historical data and then used to generate future forecasts.

Q6. How do ACF and PACF plots help in identifying the order of ARIMA models?

The ACF (Autocorrelation Function) shows correlation between a time series and its lagged values. The PACF (Partial Autocorrelation Function) measures the correlation at a particular lag after accounting for the effects of shorter lags.

For a stationary series, the ACF and PACF provide useful clues for selecting AR and MA orders:

• A PACF that cuts off after lag p can suggest an AR(p) component.

• An ACF that cuts off after lag q can suggest an MA(q) component.

• If both tail off gradually, a mixed ARMA structure may be appropriate.

These plots are guides; final model selection should also consider diagnostics and forecasting performance.

Q7. What are the assumptions of ARIMA models, and how can they be tested for in practice?

Important assumptions include:

• After required differencing, the modeled series is approximately stationary.

• The residuals should behave approximately like white noise: little autocorrelation, stable variance, and mean close to zero.

• The relationship represented by the model is adequate for the data.

Testing can include:

• ADF or KPSS tests for stationarity.

• ACF/PACF of residuals for remaining autocorrelation.

• Ljung–Box test for residual autocorrelation.

• Residual plots to inspect variance and unusual behavior.

• Q-Q plots or other diagnostics when distributional assumptions are important.

Q8. Suppose you have monthly sales data for a retail store for the past three years. Which type of time series model would you recommend for forecasting future sales, and why?

I would first inspect the data for trend and monthly seasonality. Because three years of monthly observations contain a 12-month seasonal cycle, a seasonal model such as SARIMA would be a strong starting choice if clear seasonality is present.

SARIMA extends ARIMA by modeling seasonal effects. The seasonal period would be 12 for monthly data. A suitable model could be selected using ACF/PACF, information criteria such as AIC, residual diagnostics, and validation on a chronological holdout period.

If seasonality is weak or the data has other characteristics, alternative forecasting methods should also be compared.

Q9. What are some of the limitations of time series analysis? Provide an example of a scenario where the limitations of time series analysis may be particularly relevant.

Limitations include:

• Historical patterns may not continue into the future.

• Sudden events can make forecasts inaccurate.

• Missing values, outliers, and measurement errors can affect results.

• Long-term structural changes can reduce model reliability.

• Highly complex relationships may not be captured by a simple time series model.

• Forecast uncertainty generally increases as the forecast horizon becomes longer.

Example: forecasting retail sales during an unexpected major disruption can be difficult because historical sales patterns may no longer represent customer behavior. A model trained only on historical observations may therefore produce poor forecasts.

Q10. Explain the difference between a stationary and non-stationary time series. How does the stationarity of a time series affect the choice of forecasting model?

A stationary time series has statistical properties such as its mean and variance that are reasonably stable over time, and its dependence structure does not systematically change with time. A non-stationary series may contain trend, changing variance, or other time-dependent behavior.

Many classical models, including ARIMA's AR/MA components, work best after the relevant series has been made stationary. Differencing is one common way to remove trend and achieve stationarity. If a series is already stationary, d may be zero; if it is non-stationary, one or more differences may be required.

The stationarity assessment therefore influences model choice and preprocessing. The final choice should be supported by diagnostics and out-of-sample validation.

Google Colab / Jupyter Notebook Code

The following cells can be copied into a Google Colab notebook for basic time-series exploration and ARIMA/SARIMA modeling.

Cell 1

```
# Cell 1: Install required packages (run once in Colab)
!pip install -q pandas numpy matplotlib statsmodels
```

Cell 2

```
# Cell 2: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

Cell 3

```
# Cell 3: Load monthly sales data
# Replace the file name and column names with your own data.
df = pd.read_csv("monthly_sales.csv")

df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date").set_index("Date")

sales = df["Sales"].asfreq("MS")
print(sales.head())
```

Cell 4

```
# Cell 4: Plot the time series
sales.plot(figsize=(12, 5))
plt.title("Monthly Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.show()
```

Cell 5

```
# Cell 5: Check stationarity using ADF test
result = adfuller(sales.dropna())

print("ADF Statistic:", result[0])
print("p-value:", result[1])

if result[1] < 0.05:
    print("The series is likely stationary.")
else:
    print("The series may be non-stationary.")
```

Cell 6

```
# Cell 6: ACF and PACF
series_for_plot = sales.dropna()

fig, ax = plt.subplots(2, 1, figsize=(12, 8))
plot_acf(series_for_plot, ax=ax[0], lags=24)
plot_pacf(series_for_plot, ax=ax[1], lags=24, method="ywm")
plt.tight_layout()
plt.show()
```

Cell 7

```
# Cell 7: Example SARIMA model for monthly seasonal data
# The order and seasonal_order should be selected after diagnostics.
model = SARIMAX(
    sales,
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 12),
    enforce_stationarity=False,
    enforce_invertibility=False
)

fitted_model = model.fit(disp=False)
print(fitted_model.summary())
```

Cell 8

```
# Cell 8: Forecast the next 12 months
forecast = fitted_model.get_forecast(steps=12)
forecast_mean = forecast.predicted_mean
forecast_ci = forecast.conf_int()

ax = sales.plot(figsize=(12, 5), label="Observed")
forecast_mean.plot(ax=ax, label="Forecast")
ax.fill_between(
    forecast_ci.index,
    forecast_ci.iloc[:, 0],
    forecast_ci.iloc[:, 1],
    alpha=0.2
)

plt.title("12-Month Sales Forecast")
plt.legend()
plt.show()
```

Cell 9

```
# Cell 9: Basic chronological train/test evaluation
train = sales.iloc[:-12]
test = sales.iloc[-12:]

model = SARIMAX(
    train,
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 12),
    enforce_stationarity=False,
    enforce_invertibility=False
)
fit = model.fit(disp=False)

pred = fit.forecast(steps=12)

mae = np.mean(np.abs(test - pred))
rmse = np.sqrt(np.mean((test - pred) ** 2))

print("MAE:", mae)
print("RMSE:", rmse)
```

Note: The assignment specifically asks for a Jupyter notebook and a public GitHub repository link. The PDF contains the answers and Colab-ready code; the GitHub upload/link step must be completed by the student.